

TD2 — Les choses sérieuses commencent... doucement.

Consignes générales

Chaque exercice décrit un besoin. Il vous est demandé d'en proposer une modélisation sous forme de **diagramme de classes UML**, puis de **justifier vos choix** en quelques lignes.

Dans chaque exercice, une ou plusieurs questions portent sur une *évolution* du besoin. Traitez-les sérieusement : **la qualité d'une conception se mesure à ce qu'elle coûte quand le cahier des charges change.**

Exercice 1 — Une école de musique

Une école de musique souhaite informatiser la gestion de ses cours et de son parc d'instruments.

- Un élève est identifié par un numéro, un nom et une année d'inscription. Un professeur est identifié par un numéro, un nom et la liste des disciplines qu'il enseigne.
- L'école propose deux sortes de cours : les cours individuels, qui concernent un seul élève, et les cours collectifs, qui en concernent entre 3 et 12. Tout cours a une discipline, un professeur, une salle et un créneau hebdomadaire (jour et heure de début).
- Un cours collectif a de plus un niveau ; un cours individuel a une durée qui lui est propre.
- Une salle a un nom, une capacité, et peut être équipée d'un piano. Un cours de piano ne peut avoir lieu que dans une salle équipée.
- Une même salle ne peut pas accueillir deux cours sur le même créneau.
- L'école prête des instruments : un instrument a un numéro de série, une discipline, et est soit disponible, soit prêté à un élève. Un élève ne peut détenir qu'un instrument à la fois.
- Un professeur ne peut pas assurer deux cours sur le même créneau, et ne peut enseigner que les disciplines qui sont les siennes.

- Q1.** Proposez un diagramme de classes modélisant ce domaine : attributs, associations, multiplicités, classes abstraites.
- Q2.** Le prêt d'un instrument est-il une association ou une classe ? Les deux modélisations sont défendables : donnez-les toutes les deux et dites ce que chacune permet d'exprimer que l'autre ne permet pas.
- Q3.** Parmi les règles de l'énoncé, certaines s'expriment par une multiplicité, d'autres non. Classez-les, et proposez une manière de dire celles qui restent.
- Q4.** L'école ouvre des stages : un stage réunit plusieurs professeurs et plusieurs élèves sur quelques jours consécutifs, hors créneau hebdomadaire. Faites évoluer votre diagramme. Quelles classes existantes avez-vous dû modifier ?

Exercice 2 — Un éditeur

Un éditeur vous appelle pour concevoir son nouveau système d'information. Son catalogue est organisé en quatre niveaux.

- **Niveau 0, les offres.** Un *abonnement* a une période : il court sur un an, et la période indique le rythme des livraisons (mensuelle, trimestrielle, etc.). Un *coffret* est vendu en une seule fois et porte un taux de remise.
- **Niveau 1, les regroupements éditoriaux.** Une *collection* a un thème. Des *œuvres complètes* rassemblent la production d'auteurs ; elles portent donc le nom de ces auteurs.
- **Niveau 2, les suites d'ouvrages.** Une *série* réunit des ouvrages d'un même auteur qui se suivent : il doit être possible de retrouver le numéro d'un ouvrage au sein de sa série. Une *saga* réunit des ouvrages de plusieurs auteurs, dans un ordre de lecture conseillé.
- **Niveau 3, les ouvrages.** Un *livre* est décrit par un identifiant numérique, son auteur, son année de parution, son numéro d'édition, son format, son genre (action, polar, etc.) et son prix. Une *revue* porte en plus un numéro et une périodicité.

Un élément ne peut contenir que des éléments de niveau supérieur ou égal au sien. Un abonnement peut donc tout contenir, une collection peut contenir des regroupements, des suites et des ouvrages, et une série ne contient que des suites et des ouvrages. Tout élément, quel que soit son niveau, porte un descriptif.

Les ouvrages réunis dans un même élément ont tous le même format (poche, premium, collector, etc.). Un même ouvrage peut figurer dans plusieurs éléments, mais jamais deux fois au sein d'un même élément : deux collections peuvent contenir le même livre, mais aucun élément ne peut alors contenir ces deux collections à la fois.

On veut pouvoir demander à n'importe quel élément du catalogue, sans avoir à savoir de quoi il est fait :

- son **prix**. Celui d'un ouvrage est le sien. Celui d'une suite ou d'un regroupement est la somme des prix de son contenu. Celui d'un coffret est cette somme diminuée de sa remise, et celui d'un abonnement est cette somme divisée par le nombre de livraisons de l'année.
- la **liste des auteurs** qu'il réunit, sans répétition ;
- son **résumé**, constitué de son propre descriptif suivi des résumés des éléments qu'il contient.

Le catalogue évolue au cours du temps. Votre API doit donc offrir au client des fonctions permettant de créer un élément de chacun des quatre niveaux, et de transformer un ouvrage isolé en série dont il devient le premier tome. Ces fonctions doivent garantir la contrainte de format — il ne doit pas être possible de construire une collection premium contenant un livre de poche — ainsi que l'unicité des ouvrages au sein d'un élément. L'éditeur veut de plus disposer de plusieurs manières de constituer ses regroupements et ses offres, et pouvoir passer de l'une à l'autre en ne modifiant qu'une seule ligne de code. Enfin, l'ouverture d'un nouveau format ne doit obliger à modifier aucune classe existante.

- Q1.** Proposez un diagramme de classes encodant l'ensemble de ces données. Comment votre modèle exprime-t-il la règle des niveaux ? Discutez les conséquences de votre choix lorsqu'un cinquième niveau apparaîtra.
- Q2.** Le numéro d'un ouvrage au sein de sa série est-il un attribut de cet ouvrage ? Justifiez. Un ouvrage isolé, déjà proposé dans un abonnement, est transformé en série : qu'est-ce que cela change pour cet abonnement ?
- Q3.** L'éditeur ouvre le format audio. Identifiez la ligne unique à modifier pour changer de manière de constituer les regroupements et les offres avec ce nouveau format.

Exercice 3 — Représenter une expression, autrement

On se replace dans le contexte des exercices 4 et 5 du TD1. Après y avoir ajouté la possibilité de représenter une expression en HTML et en XML, on veut maintenant disposer également d’une représentation en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, en texte brut infixe, et en forme postfixe.

Ces représentations ne donnent pas le même symbole à une même opération. Par exemple, la division de 2 par 3 s’écrit $\frac{2}{3}$ en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$; `<div><cn>2</cn><cn>3</cn></div>` en XML et 2/3 en texte brut.

- Q1.** Proposez un diagramme de classes respectant les principes SOLID et tel que, du côté des expressions, une **unique** fonction suffise pour l’ensemble des représentations, quel que soit leur nombre.
 - Q2.** Comparez les patrons STRATEGY et VISITOR. Dans quel contexte vaut-il mieux utiliser l’un plutôt que l’autre ?
 - Q3.** Reprenez l’exercice 3 du TD1 (le système de fichiers) et proposez-en un diagramme de classes qui respecte les principes SOLID. Quel principe la solution issue de la correction enfreint-elle ? Par quoi le remplacez-vous ?
-

Exercice 4 — Le fermier

Un fermier possède plusieurs types d’animaux, et chaque animal requiert un soin particulier. On impose la structure suivante, que vous devez conserver :

- les animaux sont représentés par une **unique** classe **Animal**, dont le type est donné par un attribut de type **AnimalType** ;
- **AnimalType** est une énumération contenant **Vache**, **Mouton**, **Poule**, etc., et le nombre de types d’animaux présents à la ferme peut évoluer au cours de la conception ;
- le fermier est représenté par une unique classe **Farmer**, qui détient la liste de ses animaux et possède une fonction **takeCare(animal : Animal)**, ainsi qu’une fonction qui prend soin de tous ses animaux.

- Q1.** Proposez un diagramme de conception respectant la structure décrite et réalisant **takeCare** sans **switch** sur le type d’animal. Aucune structure de votre modèle ne doit associer un type d’animal à son soin (pas de tableaux **AnimalType** $\rightarrow$ **Care**).

On enrichit maintenant les règles de la ferme. Plusieurs soins peuvent concerner un même animal, et le soin qui s’applique ne dépend plus seulement de son type mais de l’état de la ferme : par exemple, tant que l’abreuvoir n’a pas été rempli, aucun animal n’est sorti au pré, et les vaches ne sont sorties qu’une fois tous les moutons nourris. L’ordre dans lequel les soins sont envisagés fait donc partie des règles, puisqu’un même animal, dans la même situation, peut recevoir un soin différent selon cet ordre. Et si aucun soin ne s’applique, le fermier va jouer avec son chien.

Le fermier, lui, ne connaît aucun programme de soins à l’avance : on doit pouvoir changer entièrement son programme — quels soins, et dans quel ordre — en ne modifiant qu’une seule ligne de code, et une fonction doit permettre de remplacer le programme d’un fermier. Enfin, on veut qu’à l’issue de la journée, tous les animaux aient reçu l’ensemble des soins prévus pour eux.

- Q2.** Faites évoluer votre diagramme, si besoin, pour prendre en compte ces règles.
- Q3.** Où, dans votre modèle, est-il écrit que les moutons sont nourris avant que les vaches ne sortent ?